

AI 使用说明情况（第三次挑战）

姓名：黄传鑫 学号：182500116

本文如实说明本次作业中使用AI的范围。实验结论均可由提交包中的代码、启动记录和原始结果复查。

1. 使用的AI模型与工具

工具	用途
Codex (GPT-5.6, medium)	阅读作业要求，辅助编写与检查实验脚本，排查环境和运行问题，汇总实验数据，检查报告与提交包。
PowerShell、Python及课程脚本	执行文件检查、数据汇总、结果复核和报告生成。

2. 使用的提示词摘要

- 按照《第三次挑战》的要求完成四GPU上的Ray Serve与SGLang路由实验，并先核对验收条件。
- 检查Ray Serve 2.59中自定义请求路由器接口及 choose_replicas 的返回语义。
- 建立1个head节点和4个逻辑worker节点，使4个Replica分别固定到4个节点和4个SGLang后端。
- 比较默认路由、不同 max_ongoing_requests、前缀亲和路由和自定义改进路由。
- 根据 requests.csv 统计吞吐量、缓存命中率、Prefill token、TTFT、端到端延迟和后端分布。
- 检查报告、AI使用说明和压缩包是否满足提交要求。

3. AI如何帮助我学习

AI帮助我把题目要求转化为可执行的实验步骤，并解释了Ray Serve请求路由器、Replica候选顺序、SGLang radix cache以及缓存命中字段的含义。我再通过源码、启动日志和实验结果核对这些解释。

在排错过程中，AI提供了依赖冲突、模型下载、磁盘空间、SSH连接和元数据异常的检查方向。实验结果出来后，AI协助把逐请求数据按阶段、前缀热度和后端进行汇总，使缓存收益、负载偏斜和尾延迟之间的关系更容易观察。

4. AI曾给出但未直接采用的建议

- 依赖版本：直接安装 sglang[all] 会升级大量依赖并占用较多磁盘空间。我根据实际环境锁定了PyTorch、SGLang、FlashInfer和Ray版本。
- 模型规模：课程示例中的0.6B模型在RTX 4090上过快，难以形成明显排队。我改用Qwen3-4B-Instruct-2507，并通过实测确认。
- 后端并发：AI一度把性能瓶颈归因于 max_running_requests。对32和256进行补充测试后，结果表明本负载的主要限制来自Ray Serve

Replica的请求级并发。

- 元数据：回放脚本记录了默认模型名，报告也曾误写后端并发值。我依据实际启动命令和日志修正，并在 results/target3/NOTES.md 中留痕。

5. 使用原则与作业感受说明

- AI生成或修改的代码必须经过实际运行、smoke test和结果校验后才采用。
- 报告中的实验数字均由 requests.csv、summary.json 等原始文件计算，不由AI编造。
- “作业感受” 第1问由Codex协助整理；第2、3问的观点和原始内容由本人提供，仅进行了标点和版式整理。